TP1-ASSR

Thibault Malespine

2 octobre 2026

Table des matières

I Mise en place d'un tunnel IPSec sans IKE **3**
- I Architecture réseau . . . 3
- II Ajout des SA . . . 3
- III Ajout des SP . . . 4
- IV Analyse du trafic en mode tunnel . . . 4
- V Analyse du trafic en mode transport . . . 4

II Mise en place d'un tunnel IPSec avec IKE **5**
- I Générer les certificats . . . 5
- II Tunnel host to host avec IKE . . . 5
- III Tunnel net to net avec IKE . . . 7
- IV Double tunnel avec IKE . . . 8

III Conclusion **10**

I Mise en place d'un tunnel IPSec sans IKE

I Architecture réseau

Pour la mise en place de l'architecture réseau de ce TP, nous avons réutilisé le script bash `build_architecture` fourni en annexe, qui s'occupe de construire le réseau et de faire le routage des machines.

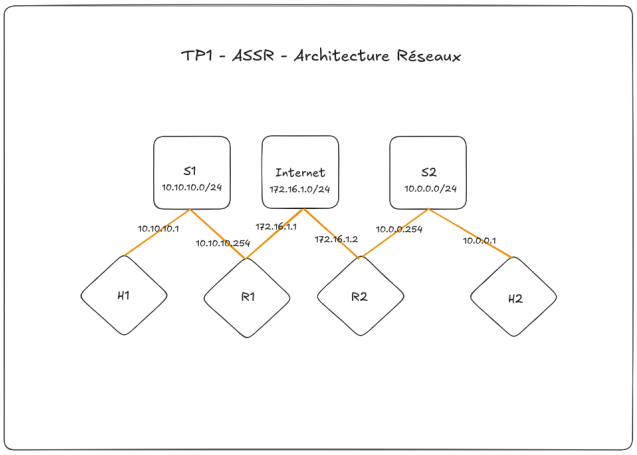


FIGURE 1 – Architecture réseau du TP

II Ajout des SA

Pour créer un tunnel IPSec, la première chose à faire est de créer des associations de sécurité (SA) qui vont définir le protocole et les algorithmes utilisés pour assurer une communication entre deux machines.

```
ip xfrm state add src 10.10.10.1 dst 10.0.0.1 proto esp spi "$SPI1"
reqid "$Rid1" mode tunnel auth sha256 "$K1" enc aes "$K2"
```

```
ip xfrm state add src 10.0.0.1 dst 10.10.10.1 proto esp spi "$SPI2"
reqid "$Rid2" mode tunnel auth sha256 "$K3" enc aes "$K4"
```

On utilise des clés en dur pour l'algorithme d'authentification (sha256) et pour celui de chiffrement (aes) utilisés par le protocole ESP.

On a besoin de 4 clés au total : 2 pour les échanges allant de h1 vers h2 et 2 pour les échanges allant de h2 vers h1.

On utilise également des identifiants : le SPI, qui sert à identifier la SA dans la SADB, et le reqid, qui permet d'associer la SA à une SP.

Ces SA sont déclarées de la même manière sur les deux machines.

III Ajout des SP

Maintenant, il faut définir les SP sur chacune des machines pour identifier les paquets qui vont transiter par le tunnel.

Sur H1, on exécute :

```
ip xfrm policy add src 10.0.0.1 dst 10.10.10.1 dir in tmpl src 10.0.0.1 dst 10.10.10.1 proto esp reqid "$Rid2" mode tunnel
ip xfrm policy add src 10.10.10.1 dst 10.0.0.1 dir out tmpl src 10.10.10.1 dst 10.0.0.1 proto esp reqid "$Rid1" mode tunnel
```

Et sur H2, on exécute :

```
ip xfrm policy add src 10.10.10.1 dst 10.0.0.1 dir in tmpl src 10.10.10.1 dst 10.0.0.1 proto esp reqid "$Rid1" mode tunnel
ip xfrm policy add src 10.0.0.1 dst 10.10.10.1 dir out tmpl src 10.0.0.1 dst 10.10.10.1 proto esp reqid "$Rid2" mode tunnel
```

IV Analyse du trafic en mode tunnel

Regardons ce que produit l'envoi d'un ping de h1 vers h2 avec notre configuration actuelle.

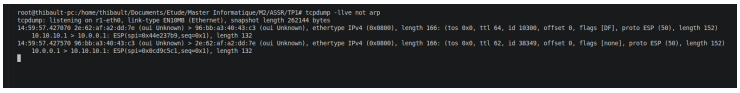


FIGURE 2 – Dump d'un ping de h1 vers h2 sur r1 en mode tunnel

On voit sur la capture d'écran que le message ICMP a été encapsulé dans un paquet ESP, ce qui correspond à l'utilisation d'IPSec.

V Analyse du trafic en mode transport



FIGURE 3 – Dump d'un ping de h1 vers h2 sur r1 en mode transport

On voit que la taille de la trame Ethernet est plus grande en mode tunnel qu'en mode transport (166 octets contre 150 octets). Cela est attendu puisqu'en mode transport seule la charge utile est chiffrée et l'en-tête IP d'origine du paquet est conservé, alors qu'en mode tunnel le paquet IP entier est chiffré et un nouvel en-tête IP est ajouté par-dessus.

II Mise en place d'un tunnel IPSec avec IKE

I Générer les certificats

Dans cette partie, nous avons mis en place un tunnel IPSec avec IKE à l'aide du paquetage strongSwan.

Pour pouvoir faire de l'authentification à l'aide du protocole IKE, il nous faut une autorité de certification capable de fournir des certificats X.509.

strongSwan nous permet de créer notre propre CA auto-signée.

Avec cette CA, on va pouvoir créer des certificats pour les machines h1 et h2, ce qui permettra l'authentification des machines lors de futures communications.

II Tunnel host to host avec IKE

Une fois les certificats créés et le fichier de configuration de strongSwan déployé pour h1 et h2, on effectue un ping de h1 vers h2 et on capture le trafic à l'aide de tcpdump sur l'interface Internet du routeur r1.

On peut alors visualiser le protocole IKE entre h1 et h2 puisqu'ils ont été configurés avec l'option `start_action = trap`, ce qui déclenche la mise en place du tunnel lorsque le premier paquet susceptible de l'utiliser est détecté.

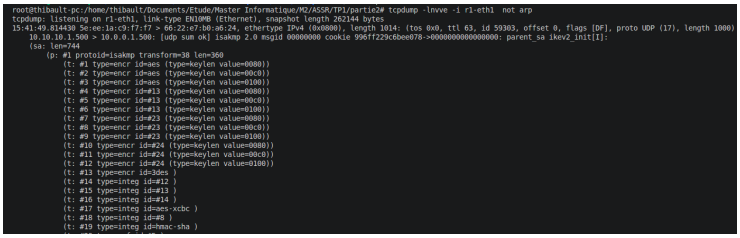


FIGURE 4 – IKE - annonce des protocoles disponibles

Sur cette première capture d'écran, on peut identifier le protocole IKEv2 (isakmp 2.0), ainsi que la liste des algorithmes supportés pour le chiffrement (encr) et l'intégrité (integ) et les groupes Diffie-Hellman supportés (dh).

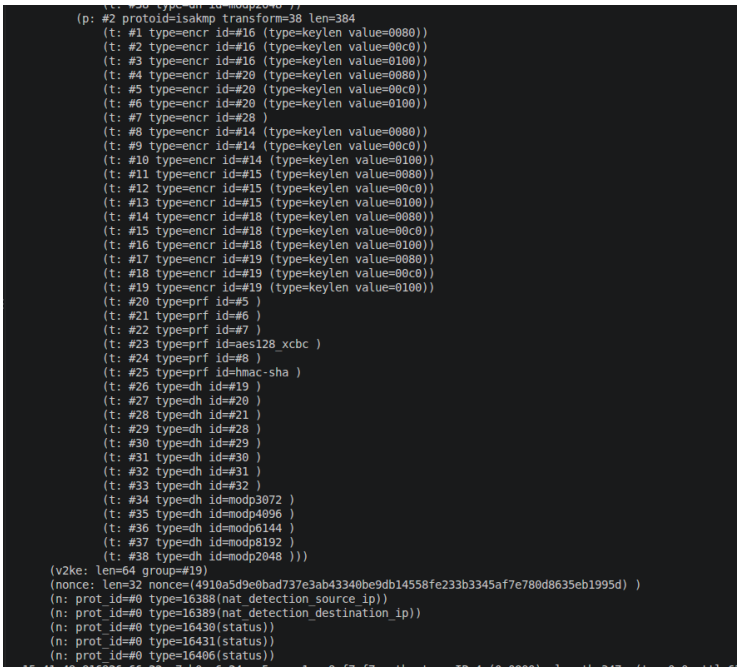


FIGURE 5 – IKE - annonce des protocoles disponibles (suite)

Cette deuxième capture d'écran montre la suite des algorithmes annoncés par h1 et également la clé publique DH de h1 (v2ke).

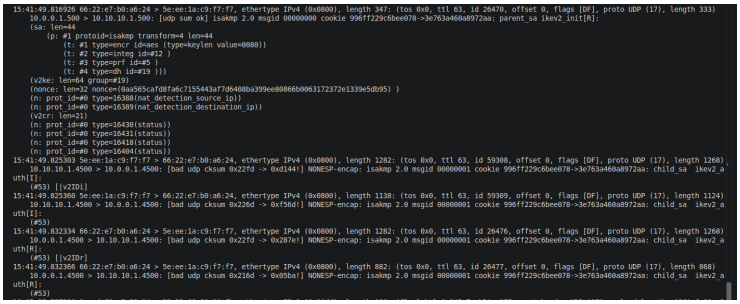


FIGURE 6 – IKE - choix des protocoles

Sur cette troisième capture d'écran, on a 5 paquets échangés. Le premier correspond à la réponse de h2, qui choisit les algorithmes à utiliser et envoie sa clé publique DH.

Les 4 paquets suivants correspondent à la phase d'authentification entre h1 et h2 et à la mise en place de la SA.

L'envoi du ping a permis la mise en place du tunnel, cependant le ping a échoué !

Un deuxième ping est nécessaire pour s'assurer que le tunnel fonctionne correctement.

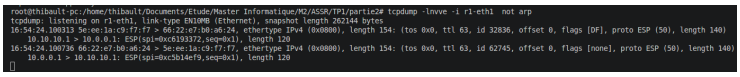


FIGURE 7 – Ping avec encapsulation ESP

On voit bien que le ping a maintenant été encapsulé avec le protocole ESP et qu'il a été correctement transmis.

III Tunnel net to net avec IKE

Dans cette section, on va mettre en place un tunnel entre r1 et r2, de la même manière que dans la partie précédente. La seule différence réside dans le fait que ce tunnel va être configuré pour des plages d'adresses IP, là où le tunnel précédent était configuré pour une adresse IP précise (tunnel point à point).

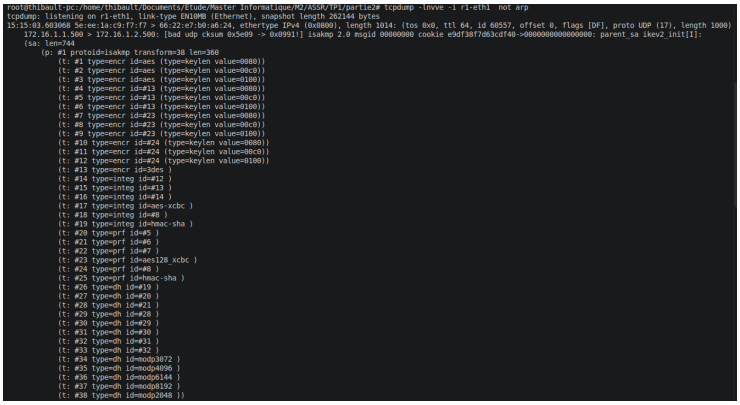


FIGURE 8 – IKE - annonce des protocoles disponibles

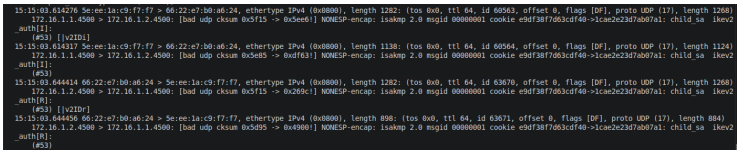


FIGURE 9 – IKE - choix des protocoles

La seule différence avec le tunnel de la partie précédente est que l'on voit maintenant que les adresses IP source et destination ont été masquées et remplacées par celles des routeurs r1 et r2.

Cela montre que ce tunnel utilise IPSec en mode tunnel et encapsule les paquets IP d'origine puis ajoute un nouvel en-tête IP, là où le tunnel précédent fonctionnait en mode transport : seule la charge utile était encapsulée.

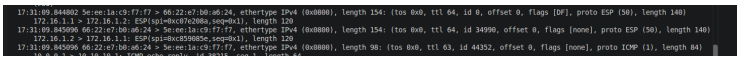


FIGURE 10 – Ping avec encapsulation ESP

Idem sur les paquets ESP : on voit que les adresses IP source et destination sont masquées.

IV Double tunnel avec IKE

En combinant le tunnel point à point entre h1 et h2 et le tunnel réseau à réseau entre r1 et r2, on obtient un double tunnel IPSec.

Regardons les échanges IKE associés.

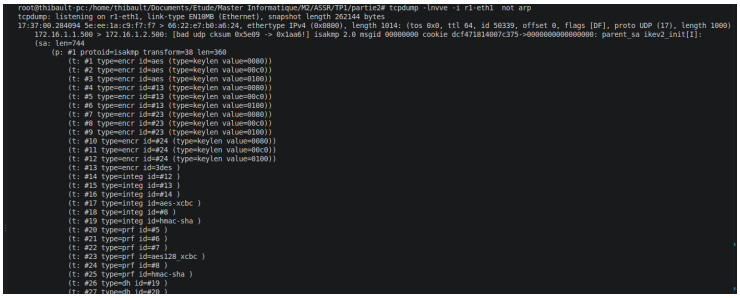


FIGURE 11 – IKE - annonce des protocoles disponibles

Comme lors de la mise en place des tunnels précédents, la négociation commence par l'annonce des protocoles cryptographiques disponibles.

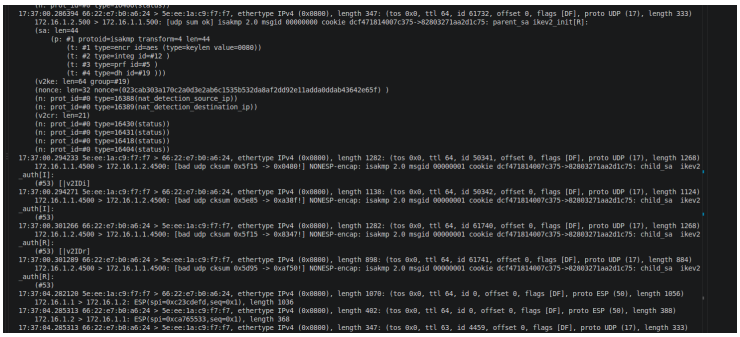


FIGURE 12 – IKE - déclaration de la SA entre r1 et r2

Ensuite, une première SA est négociée : celle du tunnel entre r1 et r2.

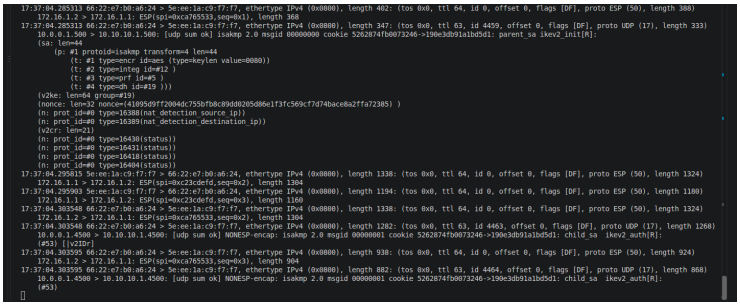


FIGURE 13 – IKE - déclaration de la SA entre h1 et h2

Puis on voit qu'une seconde SA est négociée : celle entre h1 et h2.

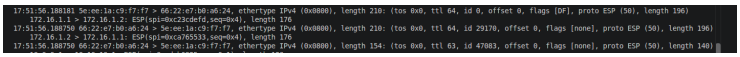


FIGURE 14 – ESP - Double encapsulation

Ensuite, lorsque l'on envoie un ping entre h1 et h2, on remarque que le paquet ESP est plus gros que précédemment (210 octets contre 154 octets), ce qui montre la double encapsulation.

III Conclusion

Ce TP nous a permis de mettre en place différentes configurations de tunnels IPSec et d'observer leur fonctionnement à l'aide de captures de trafic. La configuration manuelle des SA et des SP, sans IKE, montre le rôle de chaque élément mais impose de gérer soi-même les clés, ce qui est peu pratique et peu sûr. L'utilisation d'IKE avec strongSwan automatise la négociation des algorithmes, l'échange de clés Diffie-Hellman et l'authentification par certificats X.509. Enfin, la comparaison des modes transport et tunnel, puis la mise en place d'un double tunnel, ont mis en évidence le surcoût en taille lié à l'encapsulation ESP, en échange d'une meilleure confidentialité des adresses et des données échangées.